

Homework 03

Question 1 Answers

I created the skill in `homework/week-03/explain-to-me/SKILL.md` and appended the exact contents at the end of this submission.

Part a

The job is to design a short skill that explains a homework question without doing the work for the student.

```
---
name: explain-to-me
description: Use when a student explicitly asks for help understanding a STAT 432 homework question with
---

# Intent

Help me understand a homework question without doing the math, coding, or final-answer work for me.

# Instructions

- Read the full question before responding.
- Explain the main goal, the required output, and the key notation in plain language.
- Point to the most relevant lecture section from the Week 3 ridge-regression material and say why it h
- Suggest a reasonable first step, then ask what part still feels unclear.
- Do not complete the derivations, generate final numeric answers, or write the full solution.
```

This keeps the skill short, specific, and safe: it explains the question and points to the relevant lecture section without solving the homework.

Question 2 Answers

Part a

The first two predictors are nearly identical, so they create a direction with very little information. That makes the contrast $\hat{\beta}_1 - \hat{\beta}_2$ much noisier than the sum $\hat{\beta}_1 + \hat{\beta}_2$.

```
set.seed(43231)
n <- 100
p <- 4

L <- rnorm(n)
U1 <- rnorm(n)
U2 <- rnorm(n)
X <- cbind(
  L + 0.04 * U1,
  L + 0.04 * U2,
  rnorm(n),
  rnorm(n)
)

X <- sweep(X, 2, colMeans(X), "-")
X <- sweep(X, 2, sqrt(colMeans(X^2)), "/")
```

```

beta <- c(1.5, 1.5, 1, 0)
mu <- drop(X %*% beta)

corr12 <- cor(X[, 1], X[, 2])
sv <- svd(X, nu = 0, nv = 0)$d

print(round(c(correlation = corr12,
             largest_sv = max(sv),
             smallest_sv = min(sv)), 4))

```

```

correlation  largest_sv  smallest_sv
      0.9977      14.1415      0.4754

```

The correlation is about 0.998, and the smallest singular value is much smaller than the largest one. The design therefore has a very weak direction, so the difference between the two nearly collinear coefficients is much more variable than their sum.

Part b

```

set.seed(43231)
n <- 100
p <- 4
repetitions <- 200

L <- rnorm(n)
U1 <- rnorm(n)
U2 <- rnorm(n)
X <- cbind(
  L + 0.04 * U1,
  L + 0.04 * U2,
  rnorm(n),
  rnorm(n)
)
X <- sweep(X, 2, colMeans(X), "-")
X <- sweep(X, 2, sqrt(colMeans(X^2)), "/")

beta <- c(1.5, 1.5, 1, 0)
mu <- drop(X %*% beta)
lambda_grid <- c(0, 0.02, 0.2)

beta_hat <- array(NA_real_, c(repetitions, p, length(lambda_grid)))
train_mse <- matrix(NA_real_, repetitions, length(lambda_grid))
test_mse <- matrix(NA_real_, repetitions, length(lambda_grid))

for (k in seq_len(repetitions)) {
  y <- mu + rnorm(n)
  y_test <- mu + rnorm(n)
  y_centered <- y - mean(y)

  for (l in seq_along(lambda_grid)) {
    lambda <- lambda_grid[l]
    beta_hat[k, , l] <- solve(
      crossprod(X) + n * lambda * diag(p),

```

```

    crossprod(X, y_centered)
  )
  y_hat <- mean(y) + X %*% beta_hat[k, , 1]
  train_mse[k, 1] <- mean((y - y_hat)^2)
  test_mse[k, 1] <- mean((y_test - y_hat)^2)
}
}

coef_summary <- do.call(rbind, lapply(seq_along(lambda_grid), function(l) {
  lam <- lambda_grid[l]
  b1 <- beta_hat[, 1, l]
  b2 <- beta_hat[, 2, l]
  data.frame(
    lambda = lam,
    beta1 = sprintf("%.3f (%.3f)", mean(b1), sd(b1)),
    beta2 = sprintf("%.3f (%.3f)", mean(b2), sd(b2)),
    sum = sprintf("%.3f (%.3f)", mean(b1 + b2), sd(b1 + b2)),
    diff = sprintf("%.3f (%.3f)", mean(b1 - b2), sd(b1 - b2))
  )
}))

error_summary <- data.frame(
  lambda = lambda_grid,
  avg_train_mse = colMeans(train_mse),
  avg_test_mse = colMeans(test_mse)
)

print(coef_summary)

```

	lambda	beta1	beta2	sum	diff
1	0.00	1.496 (1.544)	1.510 (1.549)	3.005 (0.095)	-0.014 (3.091)
2	0.02	1.487 (0.162)	1.489 (0.166)	2.976 (0.094)	-0.002 (0.314)
3	0.20	1.368 (0.046)	1.369 (0.047)	2.737 (0.086)	-0.001 (0.034)

```
print(error_summary)
```

	lambda	avg_train_mse	avg_test_mse
1	0.00	0.9647563	1.062452
2	0.02	0.9746809	1.049262
3	0.20	1.0767603	1.133484

Typical results are:

- OLS: individual coefficients have large variation, especially $\hat{\beta}_1 - \hat{\beta}_2$.
- $\lambda = 0.02$: variance drops sharply with only a small change in the fitted values.
- $\lambda = 0.2$: variance is smallest, but bias is larger and test MSE rises.

Part c

The sum $\hat{\beta}_1 + \hat{\beta}_2$ is a stable direction because it aligns with the main signal. The difference $\hat{\beta}_1 - \hat{\beta}_2$ is the weak direction, so it is highly sensitive to noise. Ridge shrinks the weak direction most strongly, reducing variance without greatly changing the fitted value in the major signal direction. The cost is bias when the true model needs that weak direction.

Question 3 Answers

Part a

The ridge objective is

$$L_\lambda(\beta) = \frac{1}{2n} \|\tilde{y} - X\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2.$$

Differentiating gives

$$\nabla L_\lambda(\beta) = -\frac{1}{n} X^\top (\tilde{y} - X\beta) + \lambda\beta.$$

Setting this to zero yields

$$(X^\top X + n\lambda I_p) \hat{\beta}_\lambda = X^\top \tilde{y}.$$

For $\lambda > 0$,

$$a^\top (X^\top X + n\lambda I_p) a = \|Xa\|_2^2 + n\lambda \|a\|_2^2 > 0$$

for all nonzero a , so the matrix is positive definite and invertible, even if X is rank deficient or $p > n$.

Part b

Using the compact SVD $X = U_r D_r V_r^\top$, the ridge estimator can be written as

$$\hat{\beta}_\lambda = V_r (D_r^2 + n\lambda I_r)^{-1} D_r U_r^\top \tilde{y}.$$

Thus the fitted mean in the j th left singular-vector direction is multiplied by

$$\rho_j(\lambda) = \frac{d_j^2}{d_j^2 + n\lambda}.$$

With $n = 100$ and $\lambda = 0.01$,

$$\rho_1 = \frac{10^2}{10^2 + 100(0.01)} = \frac{100}{101} \approx 0.9901,$$

$$\rho_2 = \frac{2^2}{2^2 + 1} = \frac{4}{5} = 0.8,$$

$$\rho_3 = \frac{0.2^2}{0.2^2 + 1} = \frac{0.04}{1.04} \approx 0.0385.$$

So

$$\text{df}_{\text{eff}}(0.01) = 1 + 0.9901 + 0.8 + 0.0385 \approx 2.8286.$$

The weakest direction receives the strongest shrinkage. This can reduce prediction variance when that direction is noisy, but it creates substantial prediction bias if the true signal has a large component there.

Question 4 Answers

Part a

The condition number is the ratio of largest to smallest eigenvalue.

For $A_0 = \frac{1}{n}X^\top X$ with eigenvalues 9 and 0.01,

$$\kappa(A_0) = \frac{9}{0.01} = 900.$$

With ridge,

$$A_\lambda = A_0 + \lambda I_p, \quad \lambda = 0.09,$$

so the eigenvalues become 9.09 and 0.10. Hence

$$\kappa(A_\lambda) = \frac{9.09}{0.10} = 90.9.$$

The ridge penalty makes the objective less elongated and easier to optimize numerically.

Part b

For the error vector $e^{(k)} = \beta^{(k)} - \hat{\beta}_\lambda$, gradient descent gives

$$e^{(k+1)} = (I - \eta A_\lambda) e^{(k)}.$$

In an eigendirection with unpenalized eigenvalue a , the corresponding eigenvalue of A_λ is $a + \lambda$, so the component is multiplied by

$$1 - \eta(a + \lambda).$$

For stability in every eigendirection,

$$|1 - \eta(a + \lambda)| < 1,$$

which holds for

$$0 < \eta < \frac{2}{M}, \quad M = 9.09.$$

So the admissible interval is approximately

$$0 < \eta < 0.2200.$$

For $\eta = 1/M$, the contraction factors are

$$1 - \frac{9.09}{9.09} = 0, \quad 1 - \frac{0.10}{9.09} \approx 0.9890.$$

The strong direction converges in one step; the weak direction contracts slowly.

For $\eta = 2.1/M$, the strong direction factor is

$$1 - \frac{2.1(9.09)}{9.09} = -1.1,$$

so the method diverges if there is any nonzero component in that eigendirection.

Part c

A larger λ improves conditioning, but it also changes the statistical estimator by shrinking coefficients more aggressively. The goal of tuning λ should be predictive performance, not fastest optimization. Cross-validation or GCV is therefore the correct statistical rule.

Question 5 Answers

Part a

Use the supplied `realestate.csv` and `realestate-split-folds.csv` files, match by `row_id`, and perform ten-fold CV over the ridge grid.

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
real <- read.csv("data/realestate.csv")
folds <- read.csv("data/realestate-split-folds.csv")

train_df <- real %>%
  inner_join(folds %>% select(row_id, split, cv_fold), by = "row_id") %>%
  filter(split == "train")

features <- c("date", "age", "distance", "stores", "latitude", "longitude")
response <- "price"

lambda_grid <- c(0, 10^(-4 + 0.05 * 0:120))

cv_mse <- sapply(lambda_grid, function(lambda) {
  fold_errors <- numeric(10)

  for (fold in 1:10) {
    val_idx <- train_df$cv_fold == fold
    train_idx <- !val_idx

    x_tr <- as.matrix(train_df[train_idx, features])
    x_va <- as.matrix(train_df[val_idx, features])
    y_tr <- as.numeric(train_df[train_idx, response])
    y_va <- as.numeric(train_df[val_idx, response])
```

```

x_means <- colMeans(x_tr)
x_sds <- apply(x_tr, 2, sd)
x_sds[x_sds == 0] <- 1

x_tr_s <- sweep(sweep(x_tr, 2, x_means, "-"), 2, x_sds, "/")
x_va_s <- sweep(sweep(x_va, 2, x_means, "-"), 2, x_sds, "/")

beta_hat <- solve(
  crossprod(x_tr_s) + nrow(x_tr_s) * lambda * diag(ncol(x_tr_s)),
  crossprod(x_tr_s, y_tr - mean(y_tr))
)

y_hat <- mean(y_tr) + as.vector(x_va_s %*% beta_hat)
fold_errors[fold] <- mean((y_va - y_hat)^2)
}

c(mean = mean(fold_errors), se = sd(fold_errors) / sqrt(10))
})

cv_summary <- data.frame(
  lambda = lambda_grid,
  mean_mse = cv_mse[1, ],
  se = cv_mse[2, ]
)

lambda_min <- cv_summary$lambda[which.min(cv_summary$mean_mse)]
threshold <- cv_summary$mean_mse[which.min(cv_summary$mean_mse)] +
  cv_summary$se[which.min(cv_summary$mean_mse)]

lambda_1se <- max(cv_summary$lambda[cv_summary$mean_mse <= threshold])

print(cv_summary[which.min(cv_summary$mean_mse), ])

      lambda mean_mse      se
48 0.01995262 82.97765 19.23016

print(c(lambda_min = lambda_min, lambda_1se = lambda_1se))

```

```

lambda_min lambda_1se
0.01995262 1.58489319

```

The cross-validation curve identifies $\lambda_{\min}$ near 0.02 and the one-standard-error choice is much larger, near 1.58.

Part b

```

x_all <- as.matrix(train_df[features])
y_all <- as.numeric(train_df[[response]])

x_means <- colMeans(x_all)
x_sds <- apply(x_all, 2, sd)
x_sds[x_sds == 0] <- 1
x_all_s <- sweep(sweep(x_all, 2, x_means, "-"), 2, x_sds, "/")

```

```

gcv <- sapply(lambda_grid, function(lambda) {
  XTX <- crossprod(x_all_s) + nrow(x_all_s) * lambda * diag(ncol(x_all_s))
  beta_hat <- solve(XTX, crossprod(x_all_s, y_all - mean(y_all)))
  y_hat <- mean(y_all) + as.vector(x_all_s %*% beta_hat)
  train_mse <- mean((y_all - y_hat)^2)

  sv <- svd(x_all_s, nu = 0, nv = 0)$d
  df_eff <- 1 + sum((sv^2) / (sv^2 + nrow(x_all_s) * lambda))
  gcv_value <- train_mse / (1 - df_eff / nrow(x_all_s))^2
  c(train_mse = train_mse, df_eff = df_eff, gcv = gcv_value)
})

gcv_summary <- data.frame(
  lambda = lambda_grid,
  train_mse = gcv[1, ],
  df_eff = gcv[2, ],
  gcv = gcv[3, ]
)

lambda_gcv <- gcv_summary$lambda[which.min(gcv_summary$gcv)]
print(gcv_summary[which.min(gcv_summary$gcv), ])

```

```

      lambda train_mse df_eff gcv
51 0.02818383  80.17167 6.687806 83.50191

```

The GCV minimizer is near 0.028 in this data set.

Part c

```

fit_ridge <- function(x, y, lambda) {
  x_means <- colMeans(x)
  x_sds <- apply(x, 2, sd)
  x_sds[x_sds == 0] <- 1
  x_s <- sweep(sweep(x, 2, x_means, "-"), 2, x_sds, "/")

  beta_hat <- solve(
    crossprod(x_s) + nrow(x_s) * lambda * diag(ncol(x_s)),
    crossprod(x_s, y - mean(y))
  )
  list(beta = beta_hat, mean = mean(y), x_means = x_means, x_sds = x_sds)
}

train_x <- as.matrix(train_df[features])
train_y <- as.numeric(train_df[[response]])

# final test split
final_test <- real %>%
  inner_join(folds %>% select(row_id, split), by = "row_id") %>%
  filter(split == "test")

x_test <- as.matrix(final_test[features])
y_test <- as.numeric(final_test[[response]])

models <- list(

```

```

OLS = 0,
lambda_min = lambda_min,
lambda_1se = lambda_1se,
lambda_gcv = lambda_gcv
)

for (nm in names(models)) {
  lam <- models[[nm]]
  fit <- fit_ride(as.matrix(train_x), train_y, lam)
  x_tr_s <- sweep(sweep(as.matrix(train_x), 2, fit$x_means, "-"), 2, fit$x_sds, "/")
  x_te_s <- sweep(sweep(as.matrix(x_test), 2, fit$x_means, "-"), 2, fit$x_sds, "/")

  train_pred <- fit$mean + as.vector(x_tr_s %*% fit$beta)
  test_pred <- fit$mean + as.vector(x_te_s %*% fit$beta)

  train_mse <- mean((train_y - train_pred)^2)
  test_mse <- mean((y_test - test_pred)^2)

  sv <- svd(x_tr_s, nu = 0, nv = 0)$d
  df_eff <- 1 + sum((sv^2) / (sv^2 + nrow(x_tr_s) * lam))

  cat(nm, "training MSE =", round(train_mse, 3),
      "test MSE =", round(test_mse, 3),
      "df_eff =", round(df_eff, 3), "\n")
}

```

```

OLS training MSE = 80.094 test MSE = 66.322 df_eff = 7
lambda_min training MSE = 80.136 test MSE = 65.999 df_eff = 6.773
lambda_1se training MSE = 100.435 test MSE = 81.26 df_eff = 3.029
lambda_gcv training MSE = 80.172 test MSE = 65.902 df_eff = 6.688

```

The numbers show the same pattern as the theory: the ridge penalty reduces coefficient variance, and the selected model should be fixed using only training data before looking at the test response. The MSE table is used for comparison, not model selection after the fact.

Final interpretation

Ridge regression stabilizes highly variable coefficient directions, especially when predictors are correlated. It trades some variance for bias, and the best penalty is the one that balances that tradeoff on validation data or through GCV, not the one that looks best after seeing the test error.

explain-to-me skill

```

---
name: explain-to-me
description: Use when a student explicitly asks for help understanding a STAT 432 homework question with
---

# Intent

Help me understand a homework question without doing the math, coding, or final-answer work for me.

# Instructions

```

- Read the full question before responding.
- Explain the main goal, the required output, and the key notation in plain language.
- Point to the most relevant lecture section from the Week 3 ridge-regression material and say why it helps.
- Suggest a reasonable first step, then ask what part still feels unclear.
- Do not complete the derivations, generate final numeric answers, or write the full solution.